

変換を重ねる

曲がる境界の正体

rkskmt

1

前回の振り返り

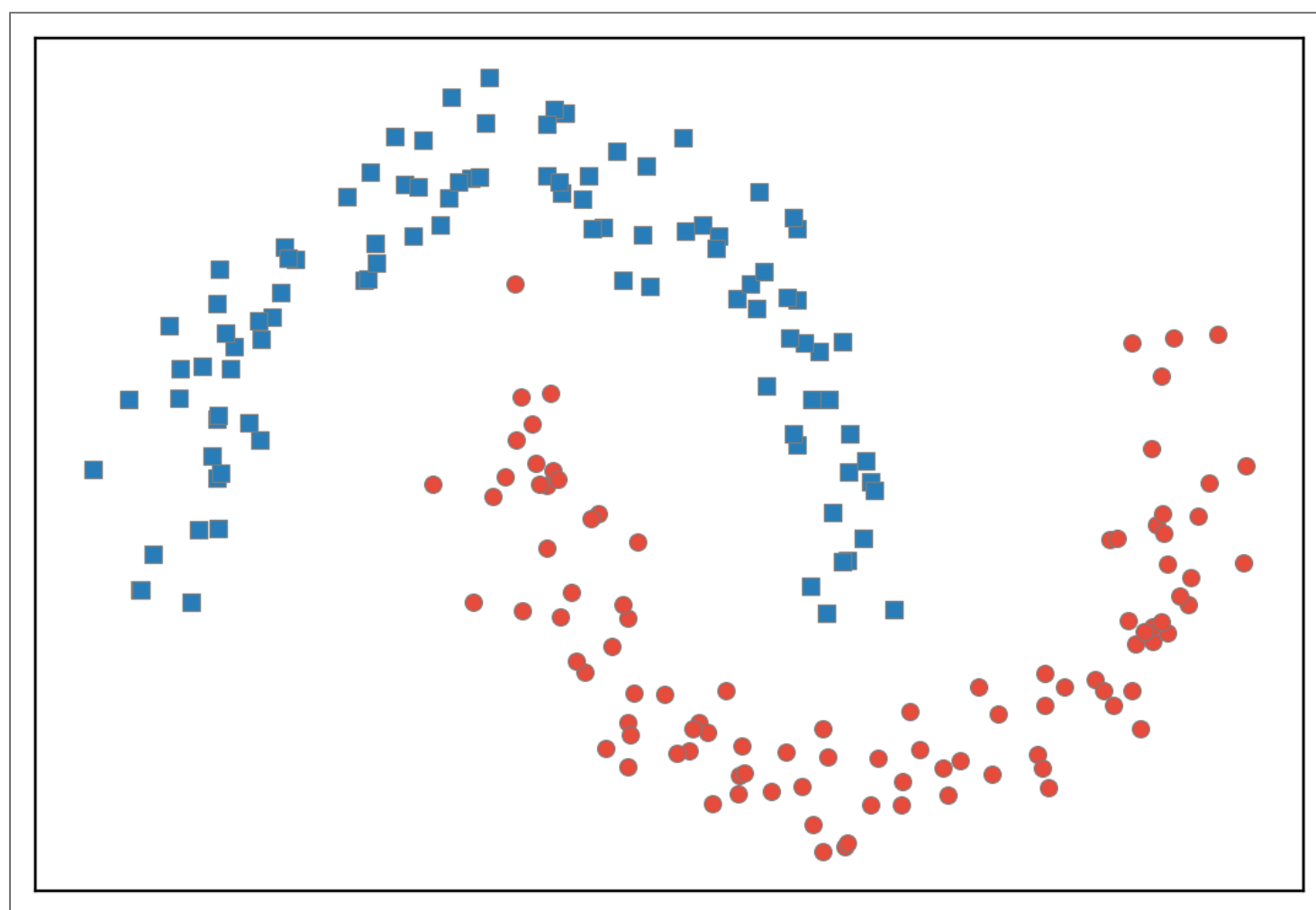
- NN の最小単位 = アフィン変換 → シグモイド (=ロジスティック回帰)
- 勾配降下法で境界が動く様子を、自分のコードで見た
- ただし、この最小単位が引けるのは **直線だけ**

📌 今回の問い

直線では絶対に分けられないデータが来たら、どうする？

2

直線では絶対に分けられないデータ



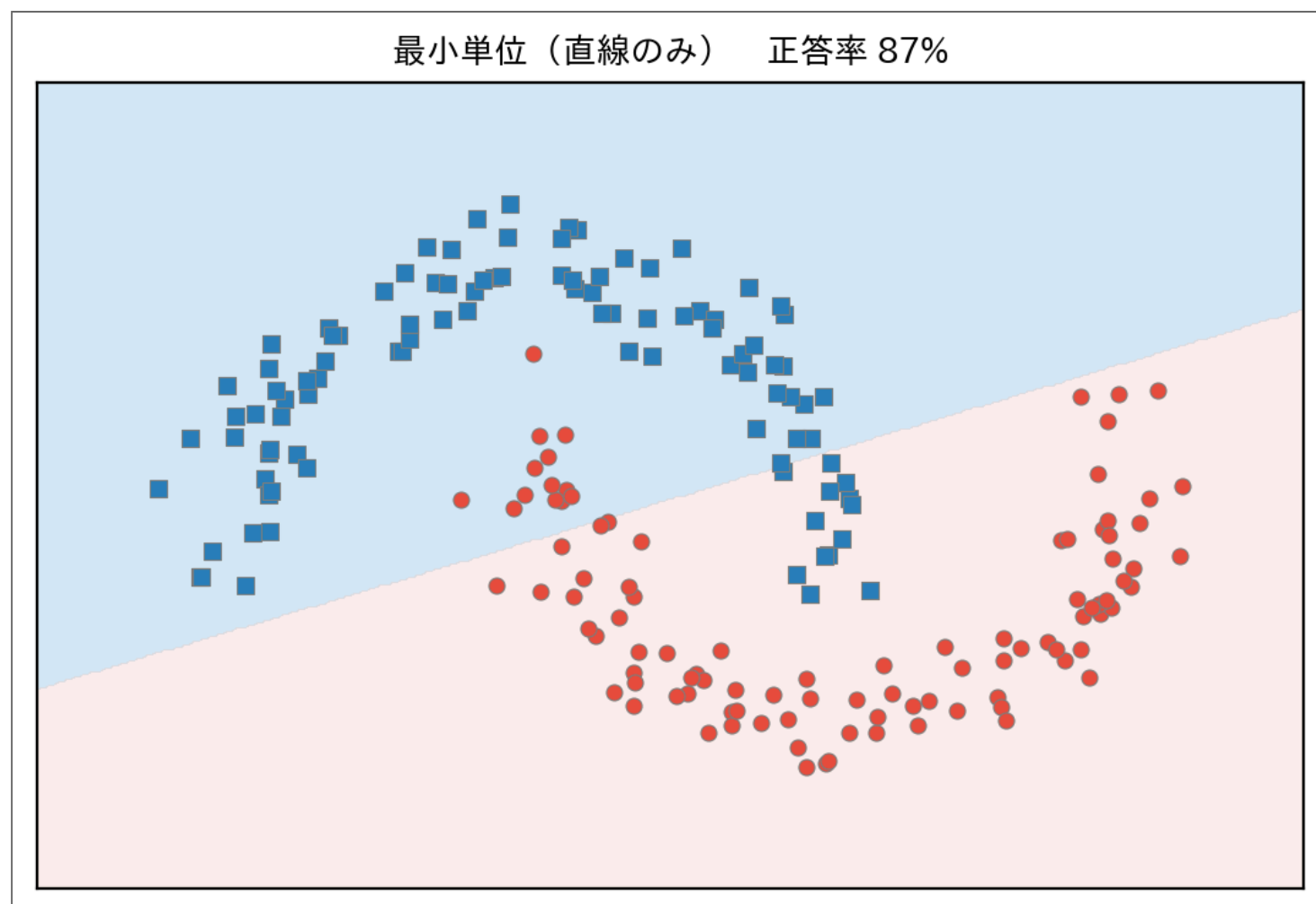
- 講義の初回に見せた、あの三日月形のデータ

📌 クイズ

前回の最小単位（直線）にこのデータを学習させたら、正答率は何%くらいになると思う？

3

最小単位の限界



- 87% で頭打ち。 **どれだけ学習しても直線は直線**
- パラメータを増やしても、形そのものを変えないと先に進めない

重ねればいい？ — 落とし穴

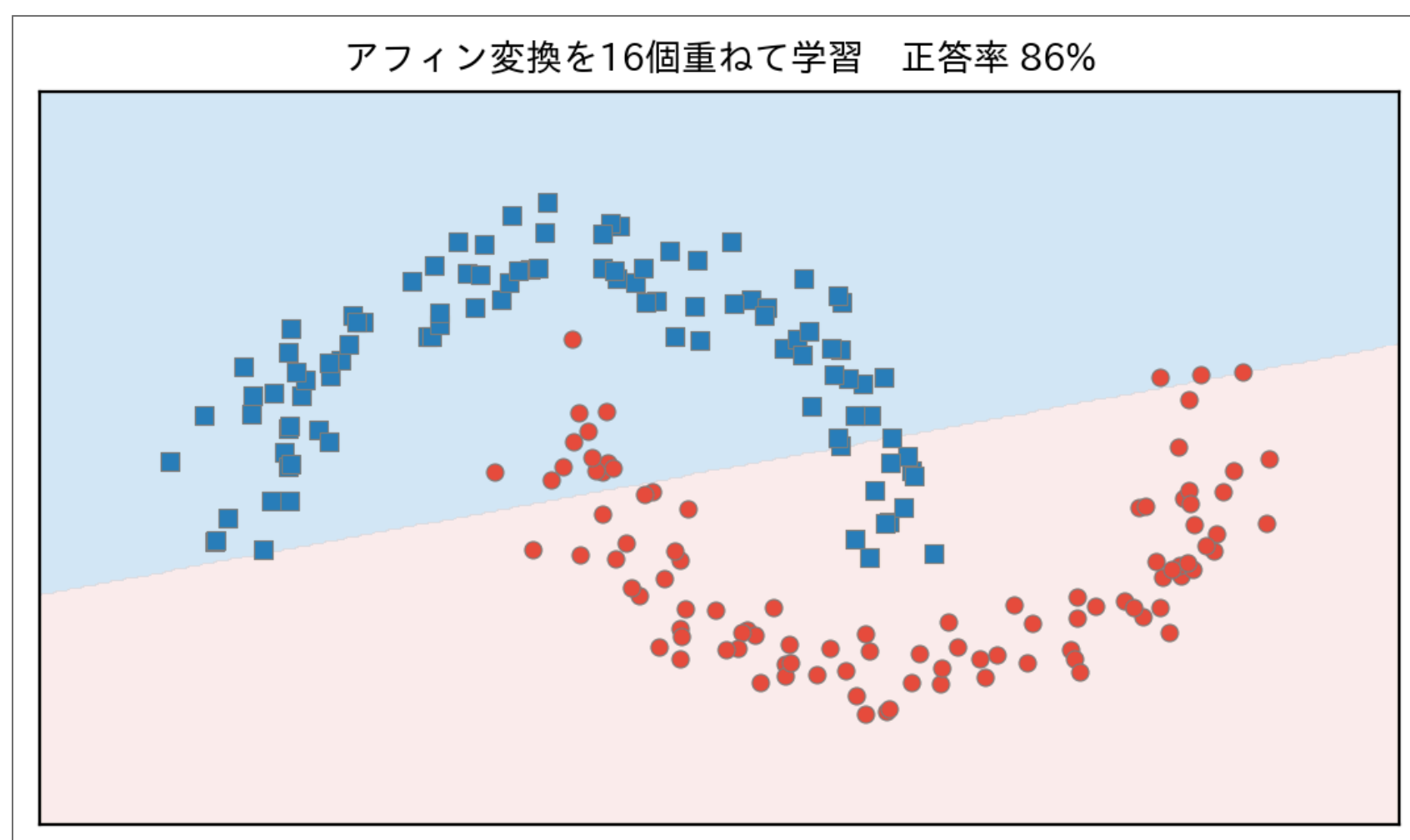
① クイズ

アフィン変換を**2回**重ねたら、曲がった境界になる？

$$z_2 = \alpha_2(\alpha_1 x + b_1) + b_2$$

$$z_2 = \underbrace{(\alpha_2 \alpha_1)}_{\text{新しい傾き}} x + \underbrace{(\alpha_2 b_1 + b_2)}_{\text{新しい切片}}$$

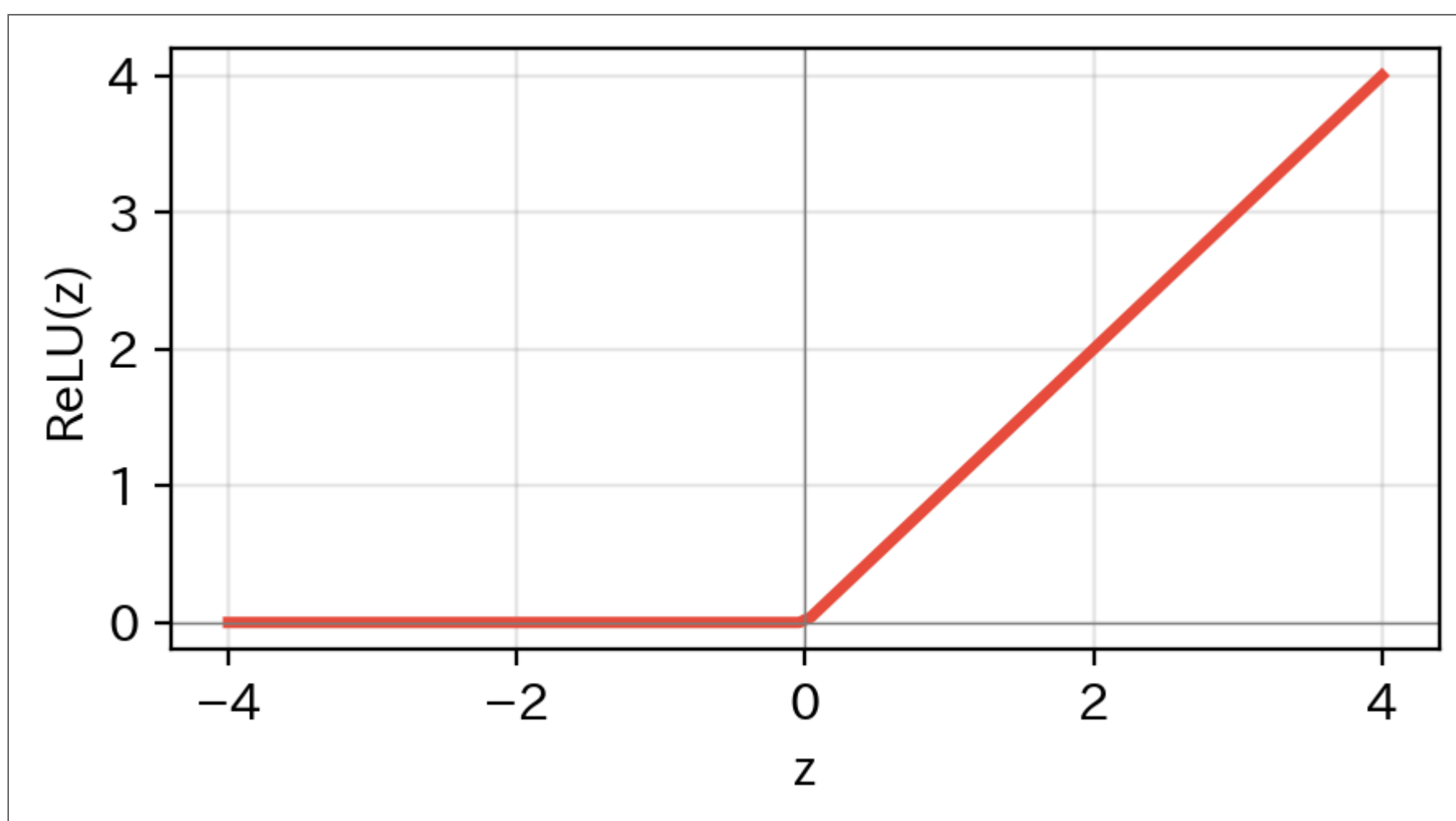
- 展開すると**ただのアフィン変換1回分**。何回重ねても同じ
- 線形（アフィン）変換は、重ねても線形のまま — **境界は直線のまま**



- 実際に16個重ねて学習させた結果 — **数式の予言どおり、直線のまま**。最小単位とほぼ同じ精度で頭打ち

間に「非線形」を挟む：ReLU

$$h = \text{ReLU}(z) = \max(0, z)$$



- 正ならそのまま、負なら 0 に潰す — それだけ
- 直線をここで**折る**ことができる（折れ曲がり＝非線形）
- アフィン変換 → ReLU → アフィン変換 と挟めば、重ねた意味が消えない

6

中間に変換を1つ挟む

$$x \xrightarrow{\alpha^T x + b} z \xrightarrow{\text{ReLU}} h \xrightarrow{wh + c} y \xrightarrow{\sigma} p$$

- パラメータは5個： α_1, α_2, b （中間）＋ w, c （出力）
- 中間の変換が「特徴を作る」、出力の変換が「まとめて判定する」
- これが**2層**のネットワーク。「層」とは変換の段数のこと

① ノート

最小単位が2個直列につながっただけ。新しい部品は **ReLU だけ**。

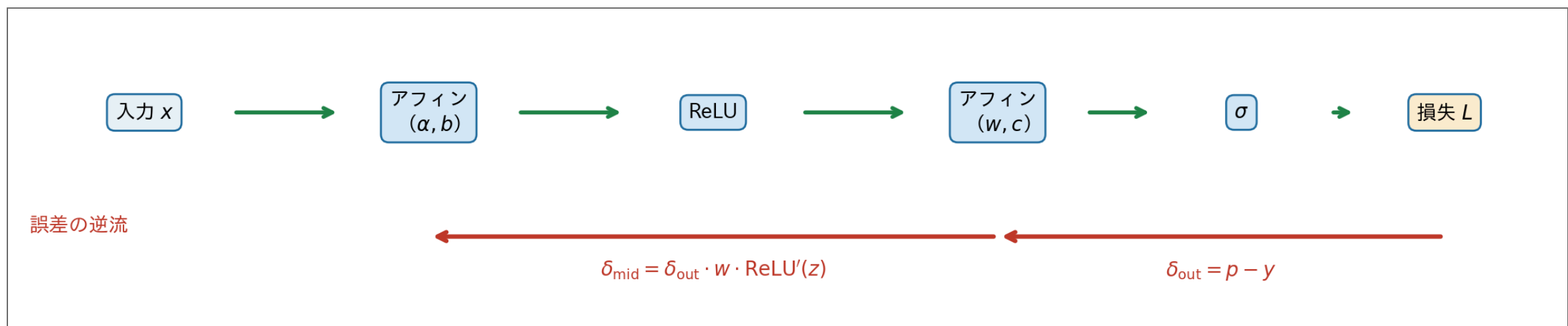
7

誤差はどうやって奥まで届くのか

- 出力側の勾配は前回と同じ： $\delta_{\text{out}} = p - y$
- 中間のパラメータには、**出力の重み w を通して** 誤差を配る：

$$\delta_{\text{mid}} = \delta_{\text{out}} \cdot w \cdot \text{ReLU}'(z)$$

- $\text{ReLU}'(z)$ は $z > 0$ なら 1、 $z \leq 0$ なら 0（折られた側には誤差が流れない）
- 各パラメータの更新は前回と同じ形： $\alpha \leftarrow \alpha - \eta \delta_{\text{mid}} x$



i 伏線

誤差が出力から入力へ**逆向きに流れていく**。この仕組みの一般形が**バックプロパゲーション**（次回、手計算で1往復する）。

8

中間の変換を2つに増やす

$$\begin{aligned} z_1 &= \alpha_1^T x + b_1, & h_1 &= \text{ReLU}(z_1) \\ z_2 &= \alpha_2^T x + b_2, & h_2 &= \text{ReLU}(z_2) \\ y &= w_1 h_1 + w_2 h_2 + c, & p &= \sigma(y) \end{aligned}$$

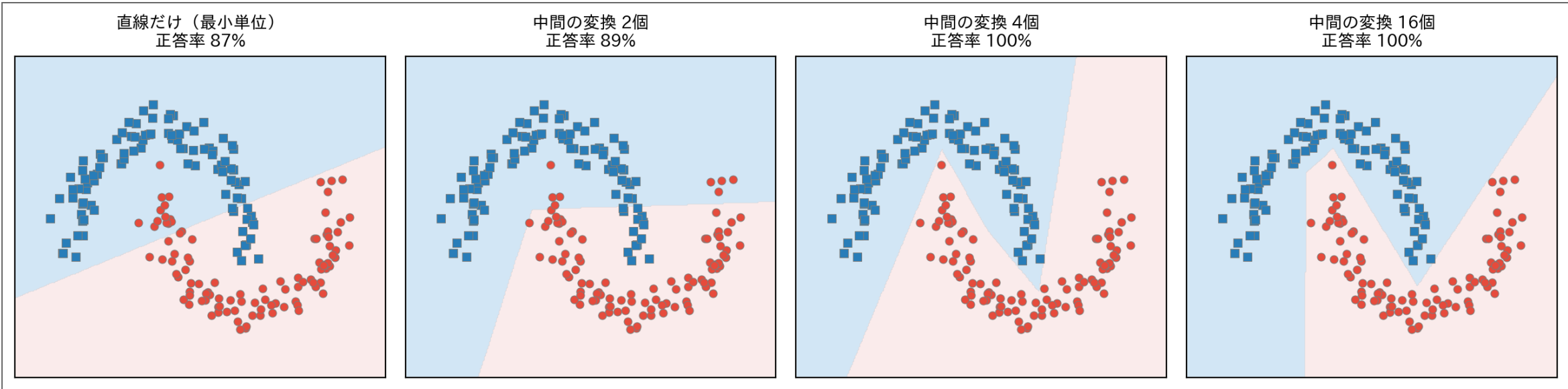
- パラメータは9個に増える
- 各変換は**独立に**自分の直線（超平面）を持ち、自分の場所で空間を折る
- 出力層は h_1, h_2 の**多数決**（線形結合）でまとめる

i ノート

「ノード」「ニューロン」と呼ばれるものの正体は、**独立した変換が複数並んでいる**だけ。

9

折り目が増えると境界が曲がる



- ReLU 1本につき **折り目が1つ** 増える
- 4本の折れ線で 100%。16本ではほぼ滑らかな曲線に見える
- 「曲がる境界」の正体は、**たくさんの折り目** だった

① 種明かし

講義の初回に見せた「機械が自力で見つけた境界」は、中間の変換16個のネットワーク。もうブラックボックスではない。

10

全部の変換が同じになってしまわないか？

① クイズ

パラメータを前回と同じように **全部 0** で初期化したら、何が起きる？

- $z_1 = z_2$ 、 $h_1 = h_2$ — 2つの変換が **完全に同じ動き** をする
- 誤差の配分 ($\delta_{\text{mid}} = \delta_{\text{out}} \cdot w \cdot \text{ReLU}'(z)$) も同じ → 更新後も同じ
- 何個並べても **実質1個** のまま (対称性の罠)
- 解決策: **ランダム初期化** で最初から全員を別の場所に置く (対称性を破る)
- 補助策: 学習中にランダムに変換を休ませる **Dropout** (依存しすぎを防ぐ)

11

出力層は問題に合わせて取り替える

- どんな NN でも、出力層は「線形結合 → 活性化関数 (activation function)」

問題	出力の活性化関数	出力の意味
2クラス分類	シグモイド	クラス1の確率 (0~1)
多クラス分類	ソフトマックス	各クラスの確率分布
回帰	なし (恒等関数)	実数値そのまま

- 中間layers は「良い特徴空間」を作ることに専念し、出力層が問題形式に合わせて答えをまとめる

12

100層でも原理は同じ

```
1 # Forward : 前から順に変換するだけ
2 for layer in layers:
3     z = W @ x + b
4     x = relu(z)
5
6 # Backward : 誤差を後ろから順に配るだけ
7 for layer in reversed(layers):
8     grad = delta * relu_grad(z)
9     W -= lr * grad @ x.T
10    b -= lr * grad
11    delta = W.T @ grad    # 1つ前の層へ
```

Python

- 層が増えても **ループが回るだけ**。各層の処理は今日やった2層と同じ
- 「ディープ」ラーニングの「ディープ」は、この段数のこと

13

まとめ

- 線形変換は **重ねても線形** — 間に ReLU を挟んで初めて重ねる意味が生まれる
- 曲がる境界の正体 = ReLU が作る **折り目の集まり**
- 変換を並べるときは **ランダム初期化** で対称性を破る
- 出力層は問題（分類・回帰）に合わせた活性化関数に取り替える

① 次回予告

層が深くなると、勾配を **全パラメータ** に届ける必要がある。今日は2層分を手で書いたが、100層では？ — 連鎖律で機械的に流す **バックプロパゲーション** へ。

14